

Basic Python Programming and Key Libraries

Advanced Computing for Biological Systems

TA: Siddhi Panday

Advanced Computing for Biological Systems

January 7, 2026

What is Python?

- High-level, interpreted programming language
- Created by Guido van Rossum (1991)
- Emphasis on code readability and simplicity
- Extensive standard library and third-party packages
- Widely used in scientific computing, data analysis, and AI/ML

Why Python for Biological Systems?

- Easy to learn and rapid prototyping
- Rich ecosystem for data analysis and machine learning
- Strong community support in bioinformatics

Basic Python Syntax

```
# Variables and data types
```

```
sequence = "ATCGATCG"
```

```
gc_content = 0.62
```

```
is_coding = True
```

```
# Lists and loops
```

```
nucleotides = ['A', 'T', 'C', 'G']
```

```
for base in nucleotides:
```

```
    print(f"Nucleotide: {base}")
```

```
# Functions
```

```
def calculate_gc(dna_seq):
```

```
    gc_count = dna_seq.count('G') + dna_seq.count('C')
```

```
    return gc_count / len(dna_seq) * 100
```

```
# Dictionary for codon table
```

```
codon_table = {'ATG': 'Met', 'TAA': 'Stop'}
```

NumPy: Numerical Python

Purpose: Foundation for numerical computing in Python

Key Features:

- N-dimensional arrays (ndarray)
- Fast vectorized operations
- Linear algebra functions
- Random number generation
- Broadcasting capabilities

```
1  import numpy as np
2
3  # Create arrays
4  gene_expr = np.array([2.5, 3.1,
5                        1.8])
6
7  matrix = np.zeros((100, 50))
8
9  # Operations
10 mean_expr = np.mean(gene_expr)
11 normalized = (gene_expr -
12               mean_expr)
13               / np.std(gene_expr)
14
15 # Random sampling
16 samples = np.random.normal(
17     loc=0, scale=1, size=1000)
```

Pandas: Data Manipulation

Purpose: Data manipulation and analysis with DataFrames

Key Features:

- DataFrames & Series
- Reading CSV, Excel, SQL
- Data cleaning & filtering
- GroupBy operations
- Time series analysis
- Missing data handling

```
1 import pandas as pd
2
3 # Read data
4 df = pd.read_csv('genes.csv')
5
6 # Data inspection
7 print(df.head())
8 print(df.describe())
9
10 # Filtering
11 high_expr = df[df['expression'] >
12                5]
13
14 # GroupBy analysis
15 grouped = df.groupby('condition')
16             .agg({'expression':
```

Matplotlib: Data Visualization

Purpose: Creating static, animated, and interactive visualizations

Key Features:

- Multiple plot types (line, scatter, bar, histogram)
- Subplots and layouts
- Customizable styling
- Publication-quality figures
- Integration with NumPy/Pandas

```
1 import matplotlib.pyplot as plt
2
3 # Line plot
4 plt.figure(figsize=(8, 6))
5 plt.plot(time, expression,
6          label='Gene A')
7 plt.xlabel('Time (hours)')
8 plt.ylabel('Expression Level')
9 plt.title('Gene Expression')
10 plt.legend()
11 plt.grid(True)
12
13 # Histogram
14 plt.hist(data, bins=30,
15          alpha=0.7)
```

PyTorch: Deep Learning

Purpose: Deep learning framework with dynamic computational graphs

Key Features:

- Tensor computation with GPU
- Automatic differentiation
- Neural network modules
- Dynamic computational graphs
- Research-friendly design
- TorchVision, TorchText

```
1 import torch
2 import torch.nn as nn
3
4 # Tensors
5 x = torch.tensor([[1, 2],
6                   [3, 4]])
7 y = x.cuda() # GPU support
8
9 # Neural Network
10 class GeneClassifier(nn.Module):
11     def __init__(self):
12         super().__init__()
13         self.fc1 = nn.Linear(100,
14                               50)
15         self.fc2 = nn.Linear(50, 2)
```

TensorFlow: End-to-End ML Platform

Purpose: Comprehensive platform for machine learning

Key Features:

- Static & eager execution
- Keras high-level API
- TensorBoard visualization
- Production deployment
- Mobile & edge support
- Distributed training

```
1 import tensorflow as tf
2 from tensorflow import keras
3
4 # Sequential model
5 model = keras.Sequential([
6     keras.layers.Dense(64,
7         activation='relu'),
8     keras.layers.Dropout(0.5),
9     keras.layers.Dense(10,
10         activation='softmax')
11 ])
12
13 # Compile
14 model.compile(
15     optimizer='adam',
16     loss='categorical_crossentropy',
```

PyTorch vs TensorFlow Comparison

Aspect	PyTorch	TensorFlow
Graph Type	Dynamic (define-by-run)	Static & Dynamic (eager mode)
Ease of Use	More Pythonic, intuitive	Steeper learning curve
Debugging	Native Python debugging	More complex debugging
Research	Highly popular in academia	Both research & production
Production	Growing deployment tools	Mature production ecosystem
Community	Strong research community	Large enterprise adoption
Mobile	Limited mobile support	TensorFlow Lite for mobile

Choice depends on: Project requirements, team expertise, deployment needs

Applications in Biological Systems

NumPy & Pandas:

- Gene expression analysis
- Sequence alignment scores
- Statistical testing
- Data preprocessing

Matplotlib:

- Heatmaps for gene expression
- Phylogenetic trees
- Protein structure plots
- Time-series of cell dynamics

PyTorch & TensorFlow:

- Protein structure prediction (AlphaFold)
- Drug discovery & molecular design
- Medical image analysis
- Genomic variant calling
- Single-cell RNA-seq analysis
- Disease prediction models
- Biological sequence modeling

These tools enable cutting-edge computational biology research!